

```
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=true
+- HashAggregate(keys=[CustomerName])
   +- Exchange hashpartitioning(CustomerName, 200)
      +- BroadcastHashJoin LeftOuter BuildRight
         :- Scan parquet ar_transactions
         +- BroadcastExchange
            +- Scan parquet customer_master
```



READ FROM THE BOTTOM UP

## DATA ENGINEERING &amp; ANALYTICS

# Global Corp.

## Reading the Plan: Spark internals, one join at a time.

A Microsoft Fabric case study in what Spark does beneath a line of PySpark: how it defers work, how it plans, how it moves data between machines, and how it can be made to read less.

### Accounts Receivable Reporting

Finance division · 10 million transactions · 5,000 customers

MICROSOFT FABRIC

# The case study, printed as a plan.

Read it the way you will read every plan in these pages: from the bottom up.  
Groundwork is the scan at the bottom. Each part builds on the one below it, and the assignment sits at the top.

== Case Study Plan ==

|     |                                                                                                               |          |
|-----|---------------------------------------------------------------------------------------------------------------|----------|
|     | <b>AS Assignment</b> (5)                                                                                      | p. 10    |
|     | Classify ten operations as narrow or wide, then tune the morning report and bring the before and after plans. |          |
| + - | <b>P3 Joins</b> (4)                                                                                           | p. 08-09 |
|     | The broadcast threshold, sort-merge and broadcast hash joins, and adaptive execution.                         |          |
| + - | <b>P2 How Spark plans</b> (3)                                                                                 | p. 06-07 |
|     | The Catalyst optimizer, predicate pushdown and column pruning, CSV versus Delta, and partition pruning.       |          |
| + - | <b>P1 How work is deferred</b> (2)                                                                            | p. 05-06 |
|     | Lazy evaluation, narrow versus wide transformations, and the shuffle.                                         |          |
| + - | <b>GW Groundwork</b> (1)                                                                                      | p. 04    |
|     | What a plan is, the three ways to print one, and how to read it from the bottom up.                           |          |

↑ START AT NODE (1) AND WORK UPWARDS

## HOW EACH CONCEPT IS BUILT

|                                                                      |                                                          |                                                            |                                                                                        |
|----------------------------------------------------------------------|----------------------------------------------------------|------------------------------------------------------------|----------------------------------------------------------------------------------------|
| <b>STORY</b><br>A moment from Aisha's week that raises the question. | <b>MECHANISM</b><br>What Spark actually does underneath. | <b>PICTURE</b><br>A diagram or plan that makes it visible. | <b>PREDICT, THEN CHECK</b><br>Guess what the plan will show, then run the named lines. |
|----------------------------------------------------------------------|----------------------------------------------------------|------------------------------------------------------------|----------------------------------------------------------------------------------------|

C05 THE SETTING

# A report that works, and an engineer who wants to know why.

Every morning Global Corp's finance division runs the accounts receivable report: yesterday's transactions joined to the customer master, so regional controllers can see who owes what. Aisha Rahman has just joined the finance data platform team. The report is not broken. But before quarter end doubles the volume, she wants to understand exactly what Spark does with it.

## WHAT THE MORNING REPORT DOES

It reads a large CSV of transactions from the lakehouse, joins each one to its customer, and sums the amounts. One line of PySpark does the join. Underneath that line, Spark decides how to read the file, how to split the work, and whether to move ten million rows across the network or not. Aisha's approach is simple: print the plan before running anything, and learn to read it.

REF-01 · THE TWO SOURCES

| SOURCE          | WHERE & SIZE                              | COLUMNS                                                                                                                   |
|-----------------|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| AR_TRANSACTIONS | Lakehouse Files, CSV.<br>10,000,000 rows. | TransactionID (int), CustomerID (int), Transaction_Date (date), TransactionAmount (double). Negative amounts are credits. |
| CUSTOMER_MASTER | Lakehouse table, Delta.<br>5,000 rows.    | CustomerID (smallint), CustomerName, Address, Credit_Limit (double).<br>One row per customer.                             |

## ABOUT THE PLANS IN THESE PAGES

The plan printed on page 4 comes from this data. Yours will carry different ID numbers after each column name, such as CustomerID#744, because Spark numbers columns afresh on every run. The nodes and their order will match.

## GW GROUNDWORK

## GROUNDWORK · WHAT A PLAN IS

## Logical and physical plans

Aisha's first instinct was to run the report and time it. Instead she typed `explain()` after the query. Nothing ran. Spark only described what it intended to do.

The description came in layers. What is each layer, and which one will the cluster actually run?

## MECHANISM

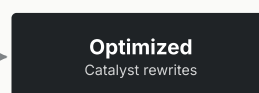
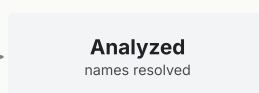
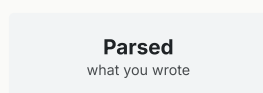
A **logical plan** is what you asked for. **Catalyst** rewrites it, then picks a **physical plan**: the steps the cluster runs.

`explain()` prints the physical plan, `explain(True)` adds the logical plans, and

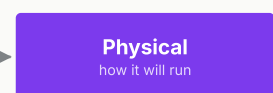
`explain(mode="formatted")` numbers each node with its details.

Fabric runs Apache Spark, so Catalyst plans every query. The Native Execution Engine, if enabled, only changes who runs the physical plan.

## LOGICAL PLANS



## PHYSICAL PLAN



`explain(True)` prints all four

`explain()` prints only this one

## PREDICT, THEN CHECK

**Predict:** which of the three versions will show you what Catalyst rewrote? **Check:** the three `q.explain` lines.

## GROUNDWORK · HOW TO READ ONE

## Read it from the bottom up

The formatted plan looked upside down to Aisha until she started reading it from the last line.

She had written two steps, a `filter` and a `select`. How many steps would the plan show?

## MECHANISM

Each indented line is a **node**; the bottom one reads the **data**. `PushedFilters` lists filters handed to the reader, `ReadSchema` the columns it reads.

`Exchange` means a **shuffle**: data moving between machines. It is the most important word in any plan.

```
== Physical Plan ==
* Filter (2)
+- Scan csv (1)

(1) Scan csv
Output [2]: [CustomerID, TransactionAmount]
PushedFilters: [NotNull(TransactionAmount), GreaterThan(TransactionAmount,5000.0)]
ReadSchema: struct<CustomerID:int,TransactionAmount:double>

(2) Filter
Condition : (isNotNull(TransactionAmount) AND (TransactionAmount > 5000.0))
```

## PREDICT, THEN CHECK

**Predict:** how many nodes will a `filter` plus a `select` produce? If a step is missing, where did it go? **Check:** the `q.explain(mode="formatted")` line.

**P1** HOW WORK IS DEFERRED

## PART I · HOW WORK IS DEFERRED

## Lazy evaluation

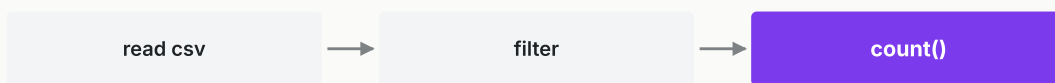
Aisha filtered ten million transactions. The cell finished almost before she let go of the keys.

Had Spark really filtered ten million rows that fast? And what would change when she asked it for a count?

**MECHANISM**

**Transformations** ( `filter`, `select`, `groupBy`, `join` ) only add to the plan. **Actions** ( `count`, `show`, `write`, `collect` ) run it.

One exception: `inferSchema` reads the file once at load time to work out the types.



TRANSFORMATIONS: ADD TO THE PLAN, RUN NOTHING

ACTION: RUNS THE PLAN

**PREDICT, THEN CHECK**

**Predict:** which of the two cells will start a Spark job, and why? **Check:** the two `chain` lines. Watch which one starts a Spark job.

## PART I · HOW WORK IS DEFERRED

## Narrow and wide transformations

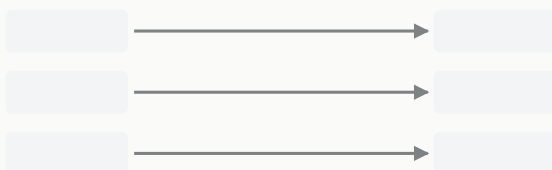
Filtering to large transactions finished quickly. Grouping the same data by customer took noticeably longer, on the same cluster.

Same data, same cluster. What made one operation so much more expensive than the other?

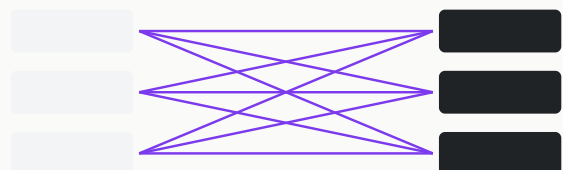
**MECHANISM**

**Narrow:** each output partition needs one input partition. Nothing moves.

**Wide:** an output partition needs rows from many partitions, so Spark shuffles. The test: could this row's answer need rows on another machine?

**NARROW · NOTHING MOVES**

each output partition comes from one input partition

**WIDE · ROWS CROSS MACHINES**

each output partition gathers rows from many

**PREDICT, THEN CHECK**

**Predict:** for each of the four operations, narrow or wide? **Check:** the four plans in a row after `chain.count()`.

## P1 · P2 HOW WORK IS DEFERRED · HOW SPARK PLANS

## PART I · HOW WORK IS DEFERRED

## The shuffle

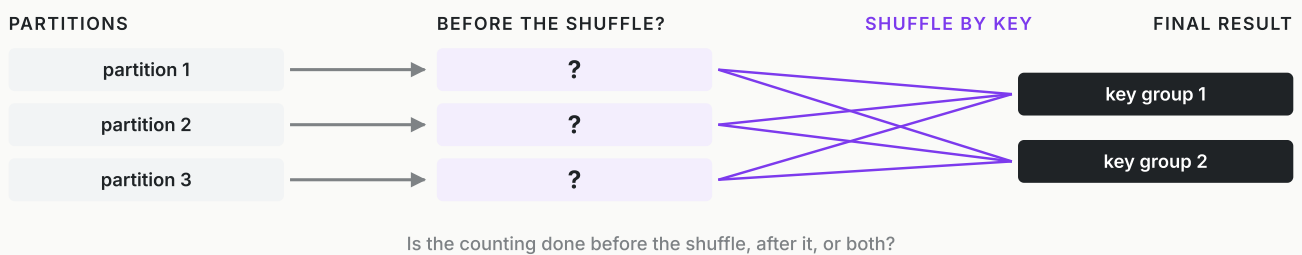
To total amounts by customer, every transaction for one customer had to end up in the same place. They started scattered across every partition of the file.

How does Spark bring them together, and could it do less work before moving anything?

**MECHANISM**

A shuffle writes rows to disk by key, sends them across the network and reads them back, so equal keys meet in one partition.

In a plan it is an `Exchange` naming its key. Aggregations can start before it, so fewer rows travel. Each shuffle also ends a stage.

**PREDICT, THEN CHECK**

**Predict:** how many aggregate steps will the group by plan have, and where will they sit around the `Exchange` ?  
**Check:** the `groupBy("CustomerID").count()` plan among those four.

## PART II · HOW SPARK PLANS

## The Catalyst optimizer

Aisha deliberately wrote her filter last: group every customer, then keep only customer 10001. As written, it looked wasteful.

Would Spark really total five thousand customers and then throw away all but one?

**MECHANISM**

You write **what** you want; Catalyst decides **how**. Its rules can move steps, drop unused columns and simplify expressions, as long as the answer stays the same.

The analyzed plan shows your order. The optimized plan shows Catalyst's.

**AS YOU WROTE IT**

```
filter(CustomerID == 10001)
```

```
groupBy + sum
```

```
read csv
```

**AFTER CATALYST**

```
?
```

```
?
```

```
?
```

Your notebook prints the right-hand side. Which step ends up where?

**PREDICT, THEN CHECK**

**Predict:** where will the filter sit in the optimized plan? **Check:** the `explain(True)` line with `10001`, then the scan's `PushedFilters`.

## P2 HOW SPARK PLANS

## PART II · HOW SPARK PLANS

## Pushdown, pruning, and why the format matters

Aisha saved a Delta copy of the transactions and ran the same query against the CSV and the copy.

Would the two plans look different? And would the two scans do the same amount of work?

## ROW FORMAT · CSV



one record per line of text

## MECHANISM

**Predicate pushdown** hands the filter to the reader.

**Column pruning** reads only the columns needed.

A CSV stores each record as a line of text. Delta stores Parquet files, which keep each column separately with its minimum and maximum per block.

## COLUMNAR · PARQUET INSIDE DELTA



each column stored separately, with min and max stats

## PREDICT, THEN CHECK

**Predict:** will the CSV and Delta plans look different? Will the scans read the same amount? **Check:** the two formatted plans on `txn_df` and `txn_delta_df`, then **size of files read** in the Spark UI.

## PART II · HOW SPARK PLANS

## Partition pruning

Most controllers only ever look at the current year. Aisha rewrote the transactions as a Delta table with one folder per year.

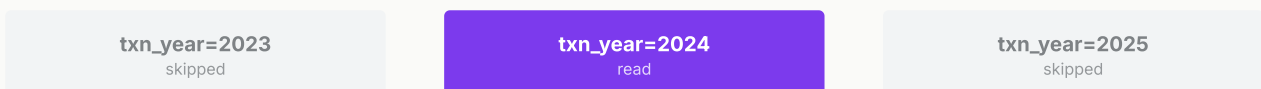
If she now filtered on a single year, how many folders would Spark open, and where would the plan show it?

## MECHANISM

`partitionBy("txn_year")` writes one folder per value. A filter on that column lets Spark skip the other folders unopened.

It only helps queries that filter on that column. Pick one with few values, such as a year; dates or customer IDs would create thousands of tiny folders.

## AR\_Transactions\_By\_Year · ONE FOLDER PER YEAR



A filter on `txn_year` only needs the folders that can match.

## PREDICT, THEN CHECK

**Predict:** which line of the scan node will name the year, and will there still be a separate filter step? **Check:** the `AR_Transactions_By_Year` lines.

## P3 JOINS

## PART III · JOINS

## How Spark picks a join strategy

Before running the report's join, Aisha wanted to know which join strategy Spark would choose, and who, or what, makes that choice.

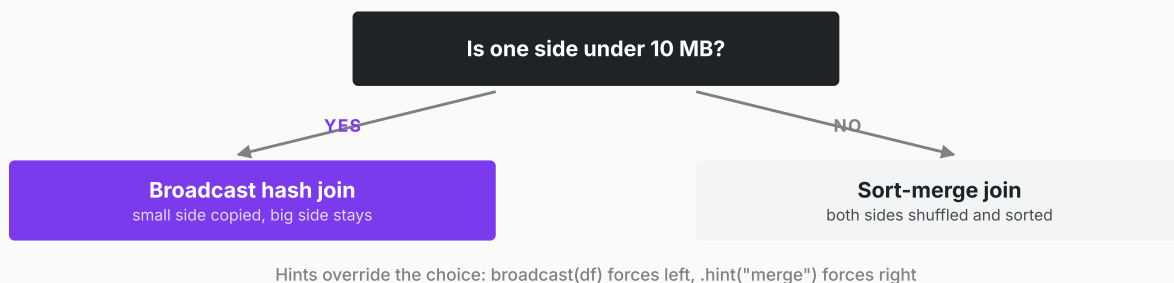
She had not asked for anything special. Would Spark pick a strategy on its own?

### MECHANISM

If one side is under

`spark.sql.autoBroadcastJoinThreshold`, Spark uses a **broadcast hash join**; otherwise a **sort-merge join**.

Hints override it: `broadcast(df)` forces broadcast, `df.hint("merge")` forces sort-merge. The plan names the strategy used.



### PREDICT, THEN CHECK

**Predict:** will the default join broadcast, or sort-merge? **Check:** the threshold print and the first join plan.

## PART III · JOINS

## Sort-merge join

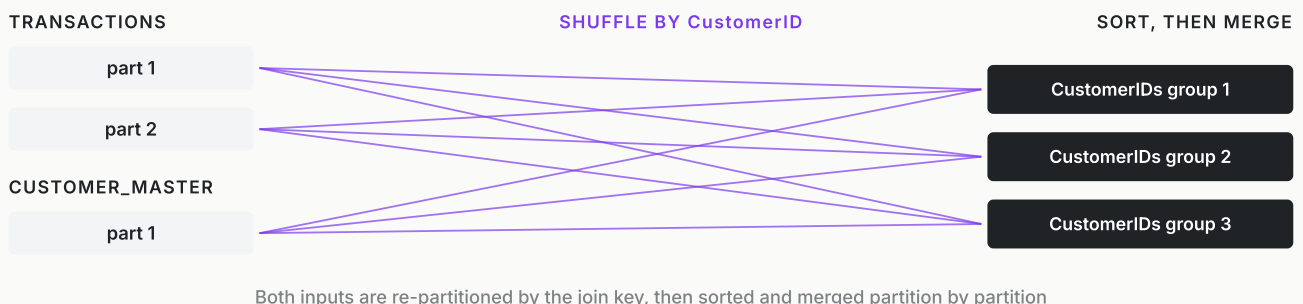
To see the alternative, Aisha forced a sort-merge join with a hint and printed the plan again.

What new steps would appear, and which of the two tables would have to move?

### MECHANISM

Both inputs are re-partitioned by the join key, so matching keys meet. Each partition is sorted and the two sides merged in order.

If the key types differ, Spark converts one side first. It suits two large tables; whether it suits one tiny one is for you to judge.



### PREDICT, THEN CHECK

**Predict:** which key will each `Exchange` partition by? Look closely at the customer side. **Check:** the `merge_join_df` plan.

## P3 JOINS

## PART III · JOINS

## Broadcast hash join

Next she forced the opposite: a broadcast join. Then she timed the two strategies side by side.

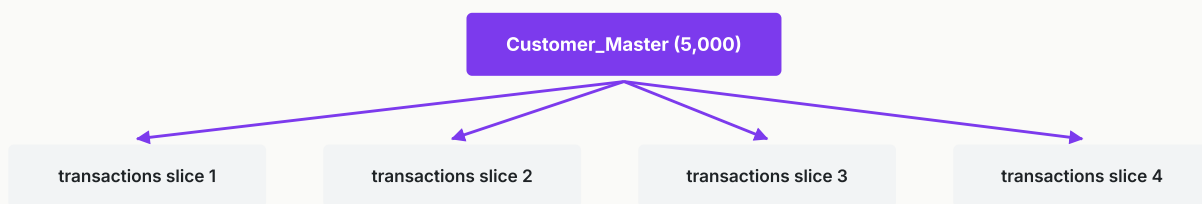
Which one would finish first, and what would the two plans say about why?

## MECHANISM

A full copy of the small table goes to every executor. Each task joins its own slice of the big table with a hash lookup, so the big side never moves.

The cost is memory: every executor must hold the whole small table.

## BROADCAST · ONE SMALL TABLE, COPIED TO EVERY EXECUTOR



Each task joins its own slice locally. The 10 million rows never move.

## PREDICT, THEN CHECK

**Predict:** which join will finish first, and which line of each plan explains the gap? **Check:** the two `count()` lines and the run time under each cell.

## PART III · JOINS

## Adaptive Query Execution (AQE)

The plan for the customer totals promised a fixed number of shuffle partitions. Aisha ran the query, then printed the plan of the same DataFrame again.

Would the plan that actually ran be the same as the plan she read beforehand?

## MECHANISM

AQE re-plans at each shuffle using real data sizes: it can **coalesce** small partitions, **switch** a sort-merge to broadcast, and **split** skewed partitions. It is on by default in Fabric.

So the plan printed before a run is not always the plan that runs. Its top line says which one you are reading.

## AQE RE-PLANS AT EVERY SHUFFLE



`explain()` before running shows only the first box. What does it show after?

## PREDICT, THEN CHECK

**Predict:** what will change at the top of the plan after the query runs, and what happens to the planned shuffle partitions? **Check:** the `totals_df` lines: plan, run, plan again.

**AS** ASSIGNMENT

# Before we tune the report.

Two tasks. Bring the results to the review. There are no answers in these pages; working them out from the plans is the point.

**ONE · CLASSIFY EACH AS NARROW OR WIDE, THEN VERIFY THREE WITH EXPLAIN()**

01 `txn_df.filter(F.col("TransactionAmount") < 0)`

02 `txn_df.groupBy("Transaction_Date").count()`

03 `txn_df.orderBy(F.col("TransactionAmount").desc())`

04 `txn_df.sortWithinPartitions("TransactionAmount")`

05 `txn_df.repartition(50)`

06 `txn_df.coalesce(1)`

07 `txn_df.union(txn_df)`

08 `txn_df.agg(F.sum("TransactionAmount"))`

09 `txn_df.join(broadcast(cust_df), "CustomerID")`

10 `txn_df.join(cust_df.hint("merge"), "CustomerID")`

**TWO · TUNE THE MORNING REPORT**

This is the report as it runs today. Read its plan first. Then make the smallest set of changes that makes it shuffle less and read less. Bring the before and after plans, not a description.

```
report_df = (txn_df.join(cust_df.hint("merge"), "CustomerID", "left")
    .filter(F.col("Transaction_Date") >= "2025-01-01")
    .groupBy("CustomerName")
    .agg(F.sum("TransactionAmount").alias("total_amount")))
```

- Your after plan uses a broadcast hash join, and the only *Exchange* left is the one for the group by.
- Your after plan reads the transactions from a Delta table, not from the CSV.
- In the before plan, the date filter already appears on the transactions scan, even though it was written after the join. Explain in one sentence why.

isFinalPlan=true

# The plan, no longer a black box.

By the end, Aisha is not guessing. She knows transformations wait and actions run, reads a plan from the bottom up, broadcasts the small side of a join, and lays the data out so Spark reads less of it.

WHAT YOU CAN NOW READ IN A PLAN

|                  |                                                |                   |                                               |
|------------------|------------------------------------------------|-------------------|-----------------------------------------------|
| Exchange         | Rows move between machines. This is a shuffle. | BroadcastExchange | The small side is copied to every executor.   |
| SortMergeJoin    | Both sides were shuffled and sorted to meet.   | PushedFilters     | The filter runs inside the scan.              |
| PartitionFilters | Whole folders are skipped before reading.      | AQEShuffleRead    | AQE resized the shuffle once it saw the data. |

DELIVER

The walkthrough notebook, run top to bottom, and your answers to both assignment tasks with the plans that support them.

EVALUATED ON

Ten operations classified, three verified with explain(), and a before and after plan that meets all three checkpoints.

NEXT

Case Study 06 keeps the same data and turns to Delta: schema enforcement, history, Time Travel, RESTORE, VACUUM, OPTIMIZE and Z-Order.

INSTRUCTOR CONTACT

Surya Kumar · [suryakumar01985@gmail.com](mailto:suryakumar01985@gmail.com)